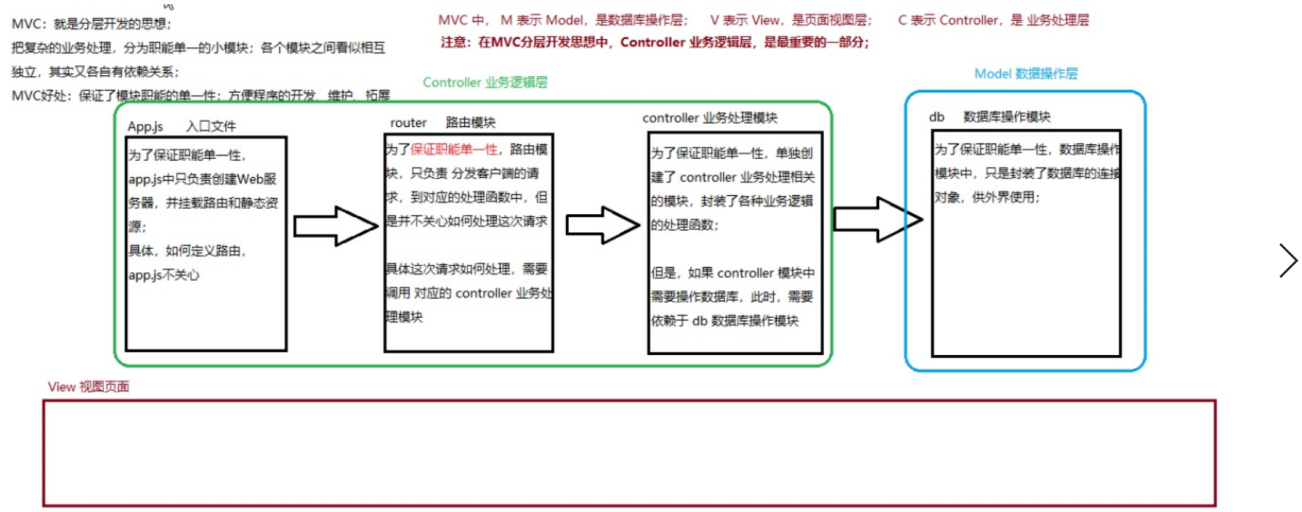
前端常见面试题总结

前端常见面试题总结

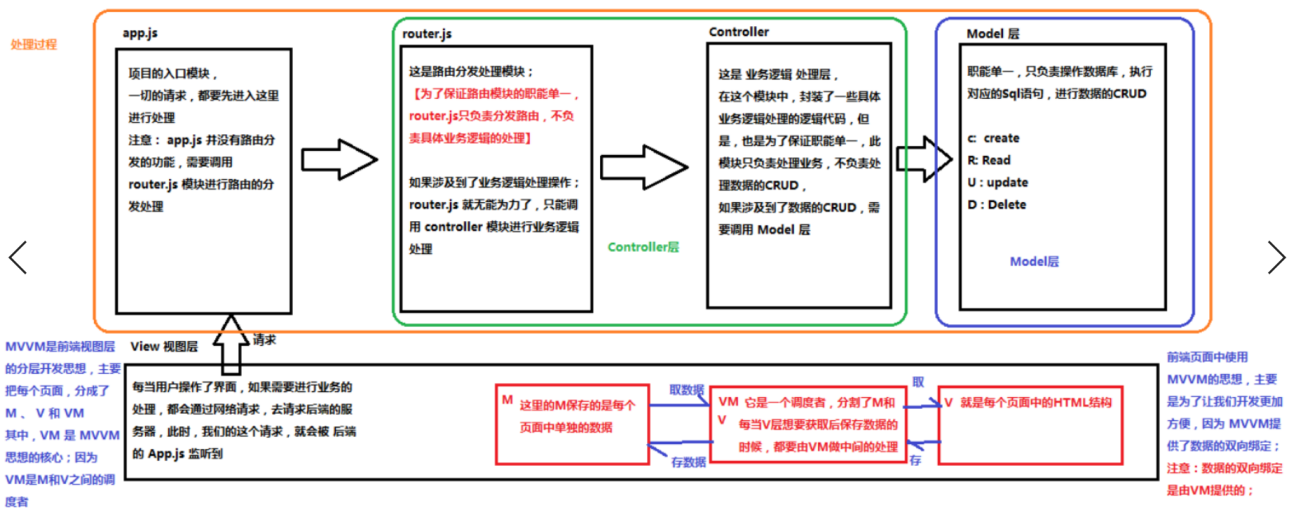
1、什么是mvvm 、 mvc 模型?
2、vue双向数据绑定的原理?
3、vue的生命周期有哪些?
4、v-if 和v-show有什么区别?
5、async await 是什么？它有哪些作用?
6、常用的数组方法有哪些?
7、数组有哪几种循环方式？分别有什么作用?
8、常用的字符串方法有哪些?
9、什么是原型链?
10、什么是闭包？手写一个闭包函数？ 闭包有哪些优缺点?
11、常见的继承有哪些?
12、后台管理系统中的权限管理是怎么实现的?
14、es6有哪些新特性?
15、v-for 循环为什么一定要绑定key ?
16、组件中的data为什么要定义成一个函数而不是一个对象?
17、常见的盒子垂直居中的方法有哪些请举例3种?
18、平时都是用什么实现跨域的?
19、cookie 、localstorage 、 sessionstrorage 之间有什么区别?
20、this 的指向有哪些?
21、什么是递归，递归有哪些优点或缺点?
22、谈谈你平时都用了哪些方法进行性能优化?
23、vue实例是挂载到那个标签上的?
24、什么是深拷贝、什么是浅拷贝?
25、js的执行机制是怎么样的?
26、请写至少三种数组去重的方法？（原生js）
27、请写出至少两种常见的数组排序的方法（原生js）
28、知道lodash吗？它有哪些常见的API ?
29、http的请求方式有哪些?
30、平时都是用那些工具进行打包的？babel是什么?
31、谈谈set 、 map 是什么?
32、清除浮动的方法有哪些?
33、常见的布局方法有哪些？他们的优缺点是什么?
34、图片懒加载是怎么实现的?
35、vue中computed 和watch 的区别是什么?
36、vue中是怎么实现父向子、子向父、兄弟之间的传值的?
37、什么vuex ,谈谈你对它的理解?
38、数据类型的判断有哪些方法？他们的优缺点及区别是什么?
39、知道symbol 吗?
40、请描述一下ES6中的class类?
41、谈谈盒子模型?
42、promise是什么？它有哪些作用?
44、箭头函数有哪些特征，请简单描述一下它?
45、移动端有哪些常见的问题，都是怎么解决的?
46、post和get 请求有哪些区别?
47、什么是同源策略?
48、http状态码分别代表什么意思?

49、BFC是什么?
50、token是什么? （加密）
51、js的数据类型有哪些?
52、**一个页面从输入 URL 到页面加载显示完成，这个过程中都发生了什么?**
53、安全问题 ： CSRF 和 XSS攻击?
54、CSRF 和 XSS 的区别
55、cookie和session 的区别
56、call、apply、bind三者的异同

1、什么是mvvm 、 mvc 模型?



MVC: MVC即model-view-controller（模型-视图-控制器）是项目的一种分层架构思想，它把复杂的业务逻辑，抽离为职能单一的小模块，每个模块看似相互独立，其实又各自有相互依赖关系。它的好处是：保证了模块的智能单一性，方便程序的开发、维护、耦合度低。



mvvm: *MVVM*：MVVM即 *Model-View-ViewModel*，(模型-视图-控制器)它是一种双向数据绑定的模式，用viewModel来建立起model数据层和view视图层的连接，数据改变会影响视图，视图改变会影响数据

2、vue双向数据绑定的原理?

vue.js 是采用数据劫持结合发布者-订阅者模式的方式，通过Object.defineProperty()来劫持各个属性的setter，getter，在数据变动时发布消息给订阅者，触发相应的监听回调。

具体步骤：

第一步：需要 observe 的数据对象进行递归遍历，包括子属性对象的属性，都加上 setter 和 getter 这样的话，给这个对象的某个值赋值，就会触发setter，那么就能监听到了数据变化

第二步：compile解析模板指令，将模板中的变量替换成数据，然后初始化渲染页面视图，并将每个指令对应的节点绑定更新函数，添加监听数据的订阅者，一旦数据有变动，收到通知，更新视图

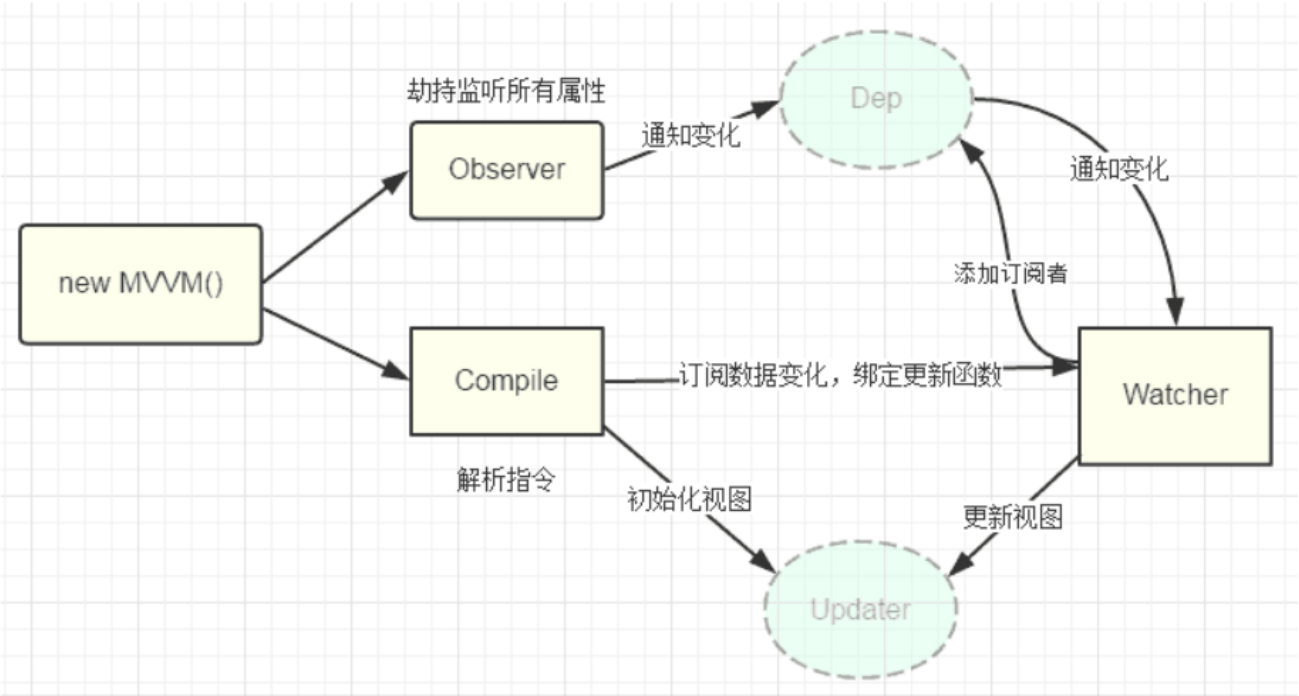
第三步：Watcher订阅者是Observer和Compile之间通信的桥梁，主要做的事情是:

1、在自身实例化时往属性订阅器(dep)里面添加自己

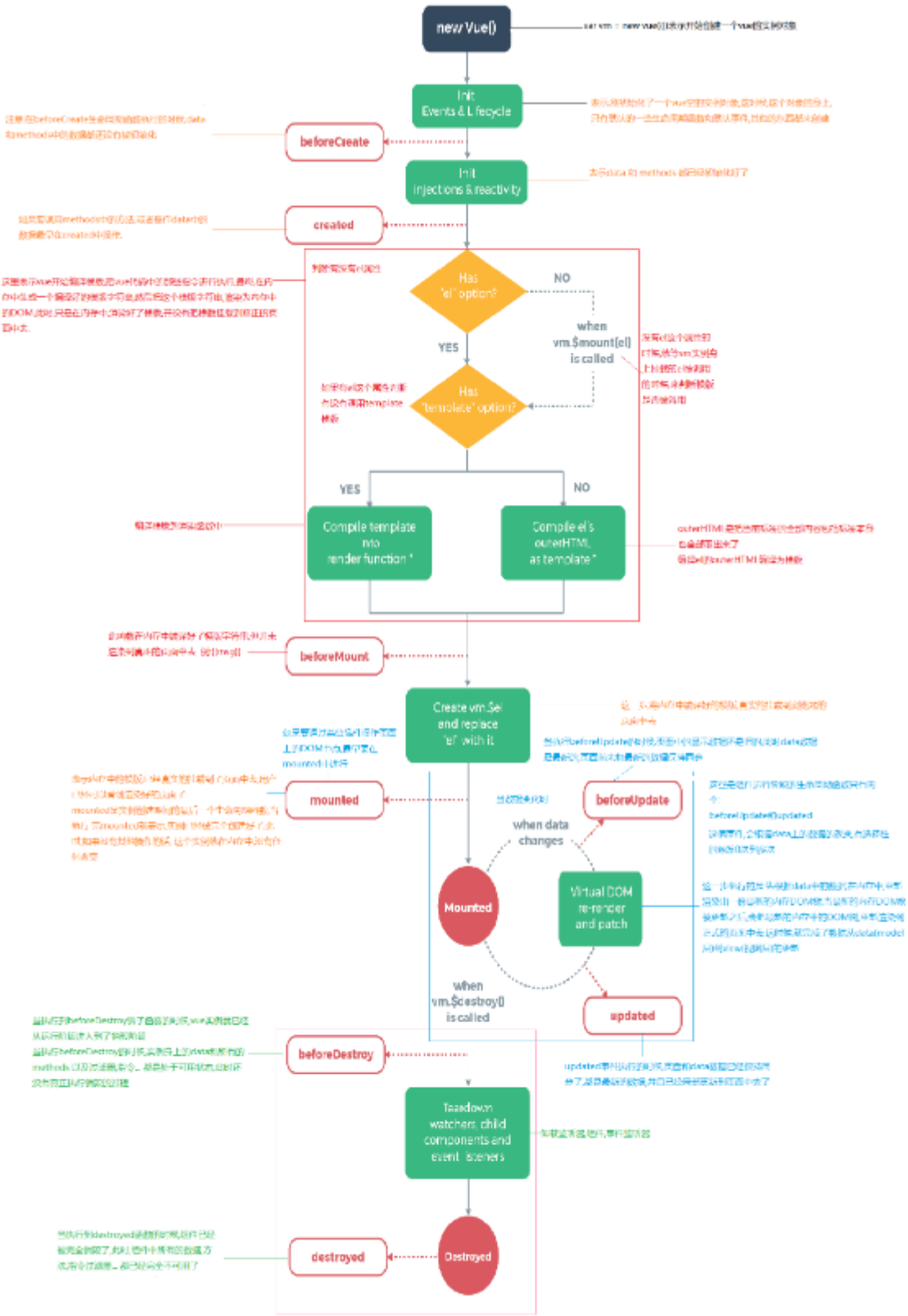
2、自身必须有一个update()方法

3、待属性变动dep.notice()通知时，能调用自身的 update() 方法，并触发Compile中绑定的回调，则功成身退。

第四步：MVVM作为数据绑定的入口，整合Observer、Compile和Watcher三者，通过Observer来监听自己的model数据变化，通过Compile来解析编译模板指令，最终利用Watcher搭起Observer和Compile之间的通信桥梁，达到数据变化 -> 视图更新；视图交互变化(input) -> 数据model变更的双向绑定效果。



3、vue的生命周期有哪些?



* template compilation is performed ahead-of-time if using a build step, e.g. single-file components

vue 实例从创建到销毁的过程就是生命周期。

也就是从开始创建、初始化数据、编译模板、挂在 dom -> 渲染、更新 -> 渲染、准备销毁、销毁在等一系列过程

vue的声明周期常见的主要分为4大阶段8大钩子函数

另外三个生命周期函数不常用

keep-alive 主要用于保留组件状态或避免重新渲染。

activated只有在keep-alive 组件激活时调用。

deactivated只有在keep-alive 组件停用时调用。

errorCapured 当捕获一个来自子孙组件的错误时被调用。此钩子会收到三个参数：错误对象、发生错误的组件实例以及一个包含错误来源信息的字符串。此钩子可以返回 `false` 以阻止该错误继续向上传播。

一、创建前 / 后

在beforeCreate生命周期函数执行的时候，data和method 还没有初始化

在created 生命周期函数执行的时候，data和method已经初始化完成

二、渲染前/后

在beforeMount 生命周期函数执行的时候，已经编译好了模版字符串、但还没有真正渲染到页面中去

在mounted 生命周期函数执行的时候，已经渲染完，可以看到页面

三、数据更新前/后

在beforeUpdate生命周期函数执行的时候，已经可以拿到最新的数据，但还没渲染到视图中去。

在updated生命周期函数执行的时候，已经把更新后的数据渲染到视图中去了。

四、销毁前/后

在beforeDestroy 生命周期函数执行的时候，实例进入准备销毁的阶段、此时data 、methods 、指令等还是可用状态

在destroyed生命周期函数执行的时候，实例已经完成销毁、此时data 、methods 、指令等都不可用

4、v-if 和v-show有什么区别?

`v-if` 是“真正”的条件渲染，因为它会确保在切换过程中条件块内的事件监听器和子组件适当地被销毁和重建，操作的实际上是dom元素的创建或销毁。

v-show 就简单得多——不管初始条件是什么，元素总是会被渲染，并且只是简单地基于 CSS 进行切换 它操作的是display:none/block属性。

一般来说，`v-if` 有更高的切换开销，而 `v-show` 有更高的初始渲染开销。因此，如果需要非常频繁地切换，则使用 `v-show` 较好；如果在运行时条件很少改变，则使用 `v-if` 较好。

5、async await 是什么？它有哪些作用?

async await 是es7里面的新语法、它的作用就是 async 用于申明一个 function 是异步的，而 await 用于等待一个异步方法执行完成。它可以很好的替代promise 中的then

`async`函数返回一个 Promise 对象，可以使用`then`方法添加回调函数。当函数执行的时候，一旦遇到`await`就会先返回，等到异步操作完成，再接着执行函数体内后面的语句。

6、常用的数组方法有哪些?

concat() 方法用于合并两个或多个数组。此方法不会更改现有数组，而是返回一个新数组。

`**find()**` 方法返回数组中满足提供的测试函数的第一个元素的值。否则返回 `undefined` 。

`**findIndex()**`方法返回数组中满足提供的测试函数的第一个元素的**索引**。否则返回-1。

`**includes()**` 方法用来判断一个数组是否包含一个指定的值，根据情况，如果包含则返回 true，否则返回 false。

`**indexOf()**`方法返回在数组中可以找到一个给定元素的第一个索引，如果不存在，则返回-1。 （通常用它判断数组中有没有这个元素）

`**join()**` 方法将一个数组（或一个类数组对象）的所有元素连接成一个字符串并返回这个字符串。如果数组只有一个项目，那么将返回该项目而不使用分隔符。

`**pop()**`方法从数组中删除最后一个元素，并返回该元素的值。此方法更改数组的长度。

`**push()**` 方法将一个或多个元素添加到数组的末尾，并返回该数组的新长度。

`**shift()**` 方法从数组中删除**第一个**元素，并返回该元素的值。此方法更改数组的长度。

unshift() 方法将一个或多个元素添加到数组的**开头**，并返回该数组的**新长度(该**方法修改原有数组**)。**

splice() 方法通过删除或替换现有元素或者原地添加新的元素来修改数组,并以数组形式返回被修改的内容。此方法会改变原数组。

由被删除的元素组成的一个数组。如果只删除了一个元素，则返回只包含一个元素的数组。如果没有删除元素，则返回空数组。

`**reverse()**` 方法将数组中元素的位置颠倒，并返回该数组。该方法会改变原数组。

`**sort()**` 方法用原地算法对数组的元素进行排序，并返回数组。默认排序顺序是在将元素转换为字符串，然后比较它们的UTF-16代码单元值序列时构建的

7、数组有哪几种循环方式？分别有什么作用?

`**every()**` 方法测试一个数组内的所有元素是否都能通过某个指定函数的测试。它返回一个布尔值。

`**filter()**` 方法创建一个新数组, 其包含通过所提供函数实现的测试的所有元素。

`**forEach()**` 方法对数组的每个元素执行一次提供的函数。

`**some()**` 方法测试是否至少有一个元素可以通过被提供的函数方法。该方法返回一个Boolean类型的值。

8、常用的字符串方法有哪些?

charAt() 方法从一个字符串中返回指定的字符。

concat() 方法将一个或多个字符串与原字符串连接合并，形成一个新的字符串并返回。

includes() 方法用于判断一个字符串是否包含在另一个字符串中，根据情况返回 true 或 false。

indexOf() 方法返回调用它的 `String` 对象中第一次出现的指定值的索引，从 `fromIndex` 处进行搜索。如果未找到该值，则返回 -1。

match() 方法检索返回一个字符串匹配正则表达式的的结果。

padStart() 方法用另一个字符串填充当前字符串(重复，如果需要的话)，以便产生的字符串达到给定的长度。填充从当前字符串的开始(左侧)应用的。 (常用于时间补0)

replace() 方法返回一个由替换值（`replacement`）替换一些或所有匹配的模式（`pattern`）后的新字符串。模式可以是一个字符串或者一个正则表达式，替换值可以是一个字符串或者一个每次匹配都要调用的回调函数。

原字符串不会改变。

slice() 方法提取某个字符串的一部分，并返回一个新的字符串，且不会改动原字符串。

`**split()**` 方法使用指定的分隔符字符串将一个 `String` 对象分割成字符串数组，以将字符串分隔为子字符串，以确定每个拆分的位置。

`**substr()**` 方法返回一个字符串中从指定位置开始到指定字符数的字符。

trim() 方法会从一个字符串的两端删除空白字符。在这个上下文中的空白字符是所有的空白字符 (space, tab, no-break space 等) 以及所有行终止符字符（如 LF，CR）。

9、什么是原型链?

每一个实例对象上有一个**proto**属性，指向的构造函数的原型对象，构造函数的原型

对象也是一个对象，也有**proto**属性，这样一层一层往上找的过程就形成了原型链。

10、什么是闭包？手写一个闭包函数？ 闭包有哪些优缺点?

闭包（closure）指有权访问另一个函数作用域中变量的函数。简单理解就是，一个作用

域可以访问另外一个函数内部的局部变量。

```
function fn() {
    var num = 10;
    function fun() {
        console.log(num);
    }
    return fun;
}
var f = fn();
f();
```

作用：延长变量作用域、在函数的外部可以访问函数内部的局部变量，容易造成内层泄露，因为闭包中的局部变量永远不会被回收

11、常见的继承有哪些?

一、原型链继承

特点：1、实例可继承的属性有：实例的构造函数的属性，父类构造函数属性，父类原型的属性。（新实例不会继承父类实例的属性！

缺点：1、新实例无法向父类构造函数传参。

2、继承单一。

3、所有新实例都会共享父类实例的属性。（原型上的属性是共享的，一个实例修改了原型属性，另一个实例的原 型属性也会被修改！）

二、借用构造函数继承

重点：用.call()和.apply()将父类构造函数引入子类函数（在子类函数中做了父类函数的自执行（复制））

特点：1、只继承了父类构造函数的属性，没有继承父类原型的属性。

2、解决了原型链继承缺点1、2、3。

3、可以继承多个构造函数属性（call多个）。

4、在子实例中可向父实例传参。

缺点：1、只能继承父类构造函数的属性。

2、无法实现构造函数的复用。（每次用每次都要重新调用）

3、每个新实例都有父类构造函数的副本，臃肿。

三、组合继承（组合原型链继承和借用构造函数继承）（常用）

重点：**结合了两种模式的优点，传参和复用**

特点：1、可以继承父类原型上的属性，可以传参，可复用。

2、每个新实例引入的构造函数属性是私有的。

缺点：调用了两次父类构造函数（耗内存），子类的构造函数会代替原型上的那个父类构造函数。

四、原型式继承

重点：用一个函数包装一个对象，然后返回这个函数的调用，这个函数就变成了个可以随意增添属性的实例或对象。object.create()就是这个原理。

特点：类似于复制一个对象，用函数来包装。

缺点：1、所有实例都会继承原型上的属性。

2、无法实现复用。（新实例属性都是后面添加的）

五、class类实现继承

通过extends 和super 实现继承

六、寄生式继承

重点：就是给原型式继承外面套了个壳子。

优点：没有创建自定义类型，因为只是套了个壳子返回对象（这个），这个函数顺理成章就成了创建的新对象。

缺点：没用到原型，无法复用。

12、后台管理系统中的权限管理是怎么实现的?

登录：当用户填写完账号和密码后向服务端验证是否正确，验证通过之后，服务端会返回一个**token**，拿到token之后（我会将这个token存贮到cookie中，保证刷新页面后能记住用户登录状态），前端会根据token再去拉取一个 **user_info** 的接口来获取用户的详细信息（如用户权限，用户名等等信息）。

权限验证：通过token获取用户对应的 权限，动态根据用户的 权限算出其对应有权限的路由，通过 **router.addRoutes** 动态挂载这些路由。

具体思路：

登录成功后，服务端会返回一个 **token**（该token的是一个能唯一标示用户身份的一个key），之后我们将token存储在本地cookie之中，这样下次打开页面或者刷新页面的时候能记住用户的登录状态，不用再去登录页面重新登录了。

ps:为了保证安全性，我司现在后台所有token有效期(Expires/Max-Age)都是Session，就是当浏览器关闭了就丢失了。重新打开游览器都需要重新登录验证，后端也会在每周固定一个时间点重新刷新token，让后台用户全部重新登录一次，确保后台用户不会因为电脑遗失或者其它原因被人随意使用账号。

用户登录成功之后，我们会在全局钩子`router.beforeEach`中拦截路由，判断是否已获得token，在获得token之后我们就要去获取用户的基本信息了

页面会先从 cookie 中查看是否存有 token，没有，就走一遍上一部分的流程重新登录，如果有token,就会把这个 token 返给后端去拉取user_info，保证用户信息是最新的。 当然如果是做了单点登录得功能的话，用户信息存储在本地也是可以的。当你一台电脑登录时，另一台会被提下线，所以总会重新登录获取最新的内容。

先说一说我权限控制的主体思路，前端会有一份路由表，它表示了每一个路由可访问的权限。当用户登录之后，通过 **token** 获取用户的 **role** ，动态根据用户的 **role** 算出其对应有权限的路由，再通过`router.addRoutes`动态挂载路由。但这些控制都只是页面级的，说白了前端再怎么做权限控制都不是绝对安全的，后端的权限验证是逃不掉的。

我司现在就是前端来控制页面级的权限，不同权限的用户显示不同的侧边栏和限制其所能进入的页面(也做了少许按钮级别的权限控制)，后端则会验证每一个涉及请求的操作，验证其是否有该操作的权限，每一个后台的请求不管是 get 还是 post 都会让前端在请求 `header`里面携带用户的 **token**，后端会根据该 **token** 来验证用户是否有权限执行该操作。若没有权限则抛出一个对应的状态码，前端检测到该状态码，做出相对应的操作。

使用vuex管理路由表，根据vuex中可访问的路由渲染侧边栏组件。

具体实现：

创建vue实例的时候将vue-router挂载，但这个时候vue-router挂载一些登录或者不用权限的公用的页面。

当用户登录后，获取用role，将role和路由表每个页面的需要的权限作比较，生成最终用户可访问的路由表。

调用router.addRoutes(store.getters.addRouters)添加用户可访问的路由。

使用vuex管理路由表，根据vuex中可访问的路由渲染侧边栏组件。

14、es6有哪些新特性?

ES6是2015年推出的一个新的版本、这个版本相对于ES5的语法做了很多的优化、例如：新增了let、const

let和const具有块级作用域，不存在变量提升的问题。新增了箭头函数，简化了定义函数的写法，同时可以巧用箭头函数的this、（注意箭头函数本身没有this,它的this取决于外部的环境），新增了promise解决了回调地域的问题，新增了模块化、利用import 、export来实现导入、导出。新增了结构赋值，ES6 允许按照一定模式，从数组和对象中提取值，对变量进行赋值，这被称为解构（Destructuring）。新增了class类的概念，它类似于对象。

15、v-for 循环为什么一定要绑定key？

页面上的标签都对应具体的虚拟dom对象(虚拟dom就是js对象), 循环中 ,如果没有唯一key , 页面上删除一条标签, 由于并不知道删除的是那一条! 所以要把全部虚拟dom重新渲染, 如果知道key为x标签被删除掉, 只需要把渲染的dom为x的标签去掉即可!

16、组件中的data为什么要定义成一个函数而不是一个对象?

每个组件都是 Vue 的实例。组件共享 data 属性，当 data 的值是同一个引用类型的值时，改变其中一个会影响其他

17、常见的盒子垂直居中的方法有哪些请举例3种?

利用子绝父相定位的方式来实现

```
#container{
    width:500px;
    height:500px;
    position:relative;
}
#center{
    width:100px;
    hight:100px;
     position: absolute;
     top: 50%;
     left: 50%;
    margin-top:-50px;
    margin-left:-50px;

}
```

利用Css3的transform，可以轻松的在未知元素的高宽的情况下实现元素的垂直居中。

```
#container{
    position:relative;
}
#center{
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
}
```

flex

```
#container{
    display:flex;
    justify-content:center;
    align-items: center;
}

#center{

}
```

18、平时都是用什么实现跨域的?

jsonp: 利用